

Machine Learning

Week 13 Lecture

Dr. Muhammad Ahmed Raza

Contents

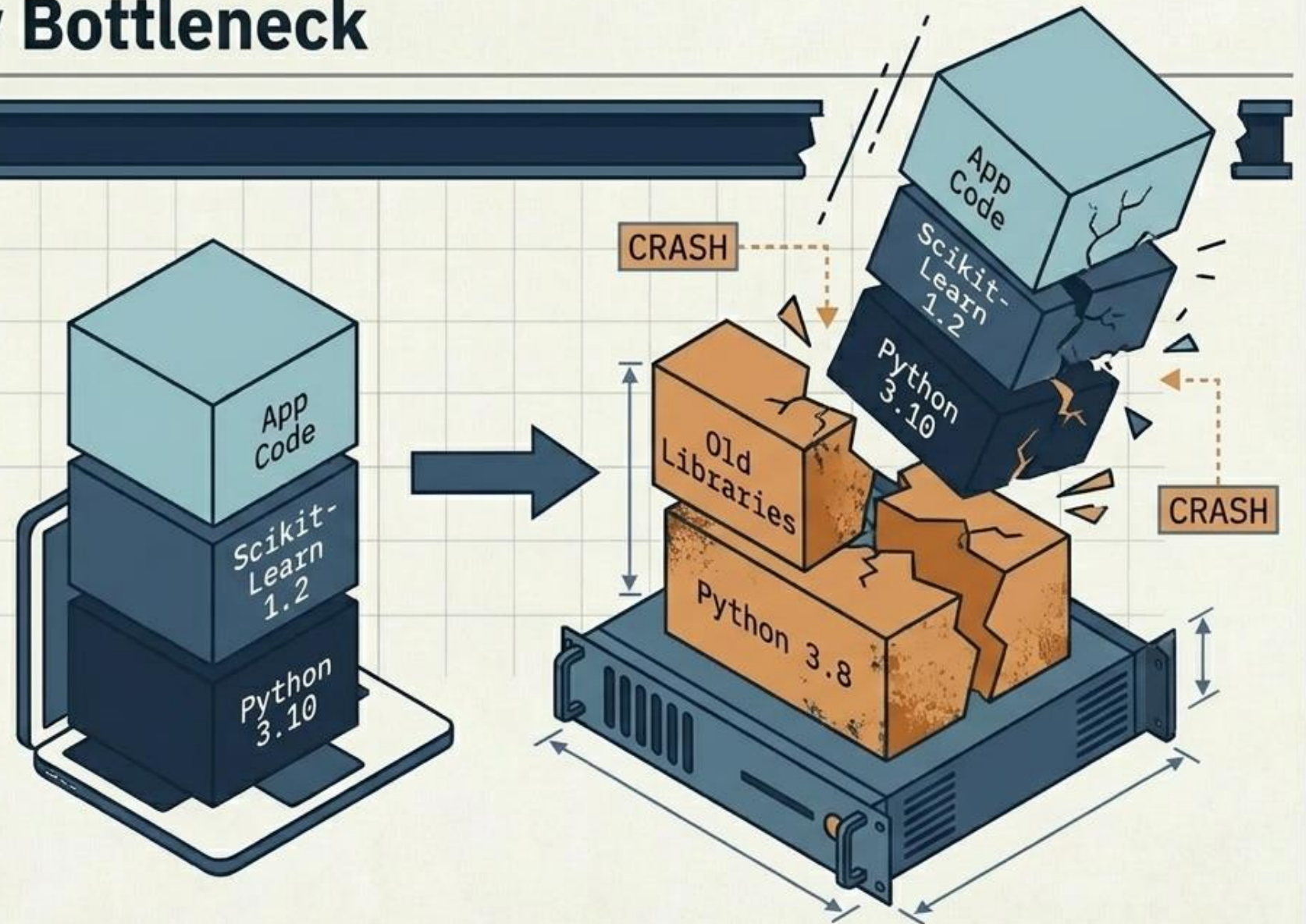
- Dependency Bottleneck
- Containers vs environment
- Drift
 - Concept vs data drift
- Statistical tests – KS test

The Dependency Bottleneck

Transporting the code is not enough. ML models rely on complex webs of specific library versions (pandas, scikit-learn, numpy).

If the host server has conflicting global packages, the application will crash.

We must isolate the environment.



Virtual Environments — venv & conda

 *Analogy: A virtual environment is like a separate room for each project — each room has its own furniture (packages), so projects never interfere with each other.*

venv (built-in)

```
# Create
python -m venv myenv

# Activate (Linux/Mac)
source myenv/bin/activate

# Activate (Windows)
myenv\Scripts\activate
```

- Built into Python 3.3+
- Lightweight, no extra install
- Python packages only
- Good for simple projects

conda (Anaconda/Miniconda)

```
# Create
conda create -n myenv python=3.10

# Activate
conda activate myenv

# Install packages
conda install numpy pandas
```

- Manages Python AND system libs
- Great for data science (numpy, CUDA)
- Heavier but more powerful
- Industry standard in DS/ML

The Anatomy of a Container

A Docker Container packages the application, dependencies, OS-level libraries, and runtime into a single, reproducible, and portable artifact.

It guarantees that if it runs locally, it will run exactly the same in the cloud.



```
COPY . /app
```

Injecting the FastAPI app and model.joblib.

```
RUN pip install -r
```

Installing exact library versions.

```
COPY requirements.txt
```

Preparing to fetch dependencies.

```
FROM python:3.9
```

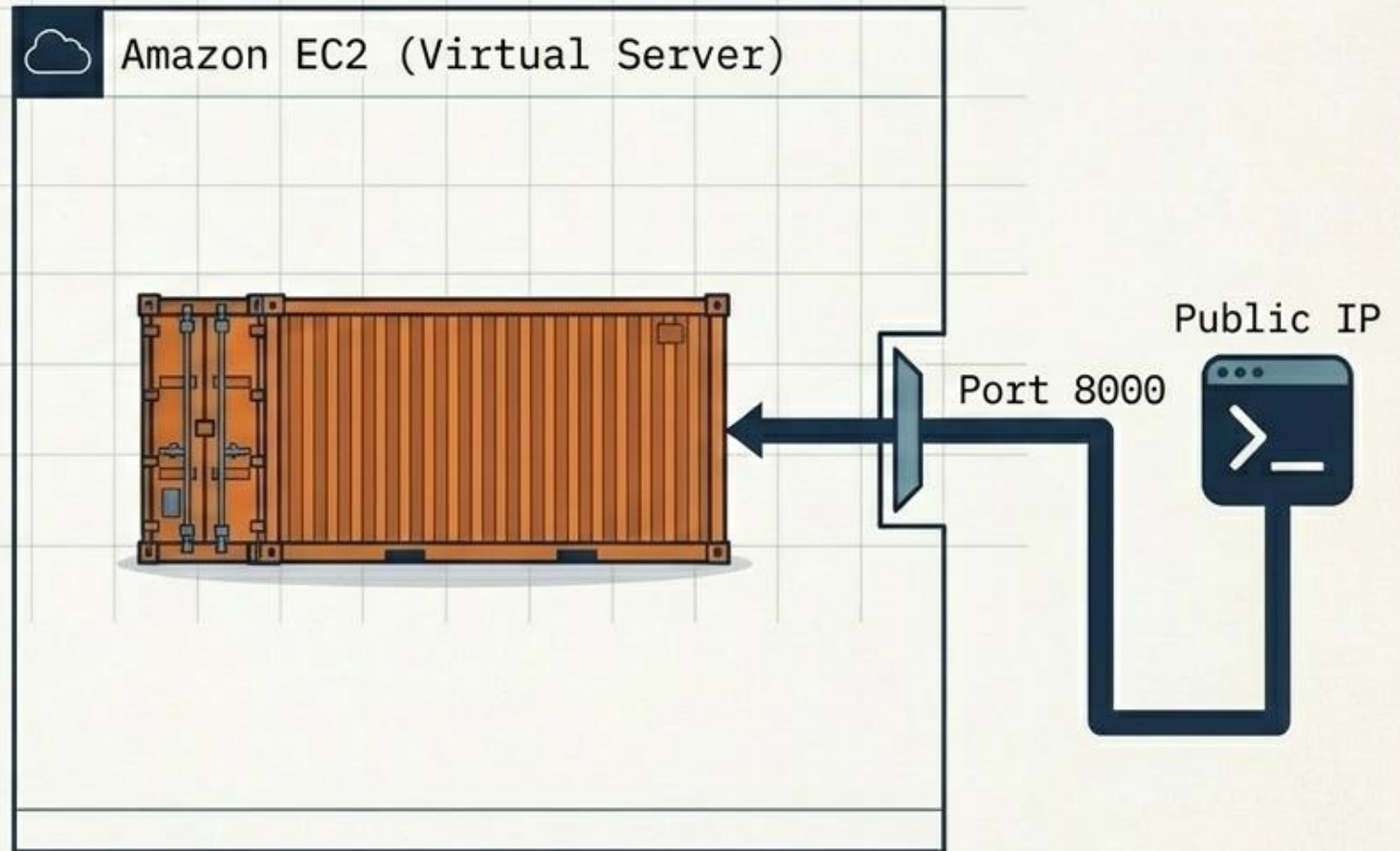
The Base OS/Runtime.

The Cloud Destination: AWS EC2

Bare Metal to Cloud: Amazon Elastic Compute Cloud (EC2) provides the virtual servers to run the Docker containers.

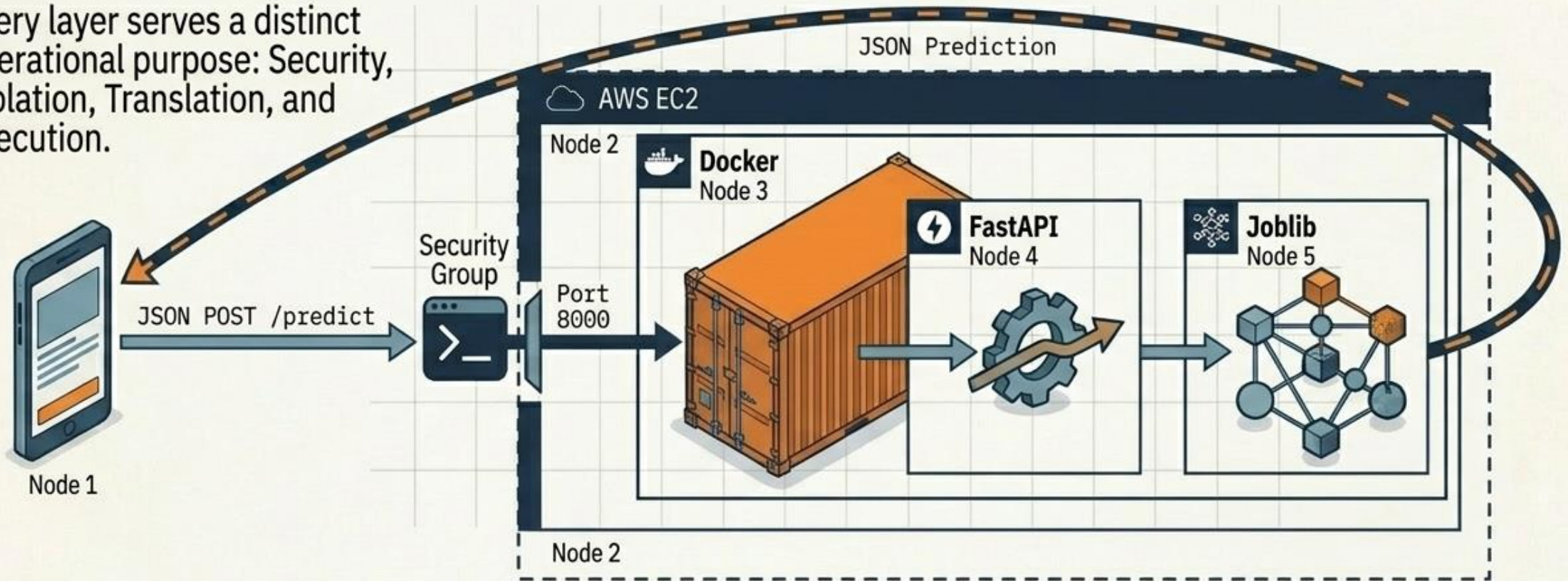
Security Groups: Network traffic must be explicitly allowed. Port 8000 (FastAPI's default) must be opened to receive incoming HTTP requests.

Advanced Scale: For high traffic, models move from standalone EC2 instances to container orchestration systems like ECS (Elastic Container Service) and ECR (Registry).



End-to-End Production Architecture

Every layer serves a distinct operational purpose: Security, Isolation, Translation, and Execution.



1. The Client

Mobile phone sends JSON POST /predict.

2. The Cloud (AWS)

Request enters an AWS EC2 instance via Public IP, passing through a Security Group gate (Port 8000).

3. The Container (Docker)

The request enters the isolated container environment.

4. The Interface (FastAPI)

Uvicorn server routes the request to the correct Python function.

5. The Brain (Joblib)

The pre-loaded serialized model evaluates the features.

6. The Return

The prediction is formatted as JSON and sent back down the chain to the phone.

Containers & Docker — Next-Level Packaging



Analogy: A Docker container is like a shipping container — it holds your app + OS + libraries, runs identically anywhere: laptop, server, cloud.



Sample Dockerfile:

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app",
    "--host", "0.0.0.0",
    "--port", "8000"]
```

⚡ Docker Commands:

```
docker build -t my-api .
```

Build the image

```
docker run -p 8000:8000 my-api
```

Run container

```
docker ps
```

List running containers

```
docker stop <id>
```

Stop container

```
docker push repo/my-api
```

Push to registry

A typical Dockerfile

- FROM: Selects the base image (e.g., FROM node:22-alpine).
- WORKDIR: Sets the working directory inside the container (e.g., WORKDIR /app).
- COPY / ADD: Transfers files from your local machine to the container's filesystem.
- RUN: Executes commands during the build process, such as installing dependencies (e.g., RUN npm install).
- EXPOSE: Informs Docker that the container listens on specific network ports at runtime.
- CMD: Provides the default command to execute when the container starts (e.g., CMD ["node", "app.js"])

venv/conda vs Docker — When to Use Each

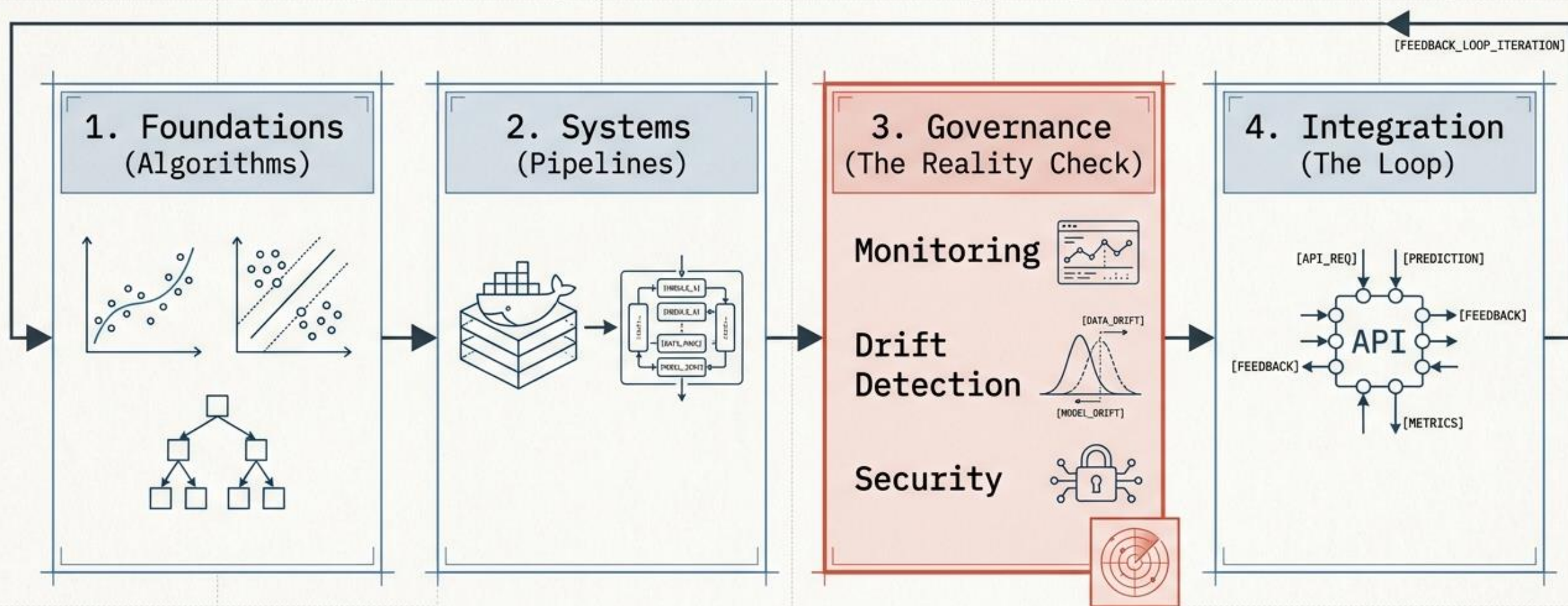
Factor	 Virtual Env (venv/conda)	 Docker Container
What it isolates	Python packages only	Python + OS + system libs
Setup complexity	Simple	Moderate learning curve
Reproducibility	Partial (OS differs)	✅ Full (same everywhere)
Portability	Same OS only	✅ Any machine / cloud
Production-ready?	Dev / simple deploys	✅ Industry standard
Startup time	Instant	Seconds
Scaling	Manual	Orchestrators (K8s, ECS)
Best for	Local dev, notebooks, quick tests	Production APIs, cloud, teams

💡 *Rule of thumb: Use venv for learning/dev, Docker for anything that goes to production.*

The Production Pipeline Blueprint

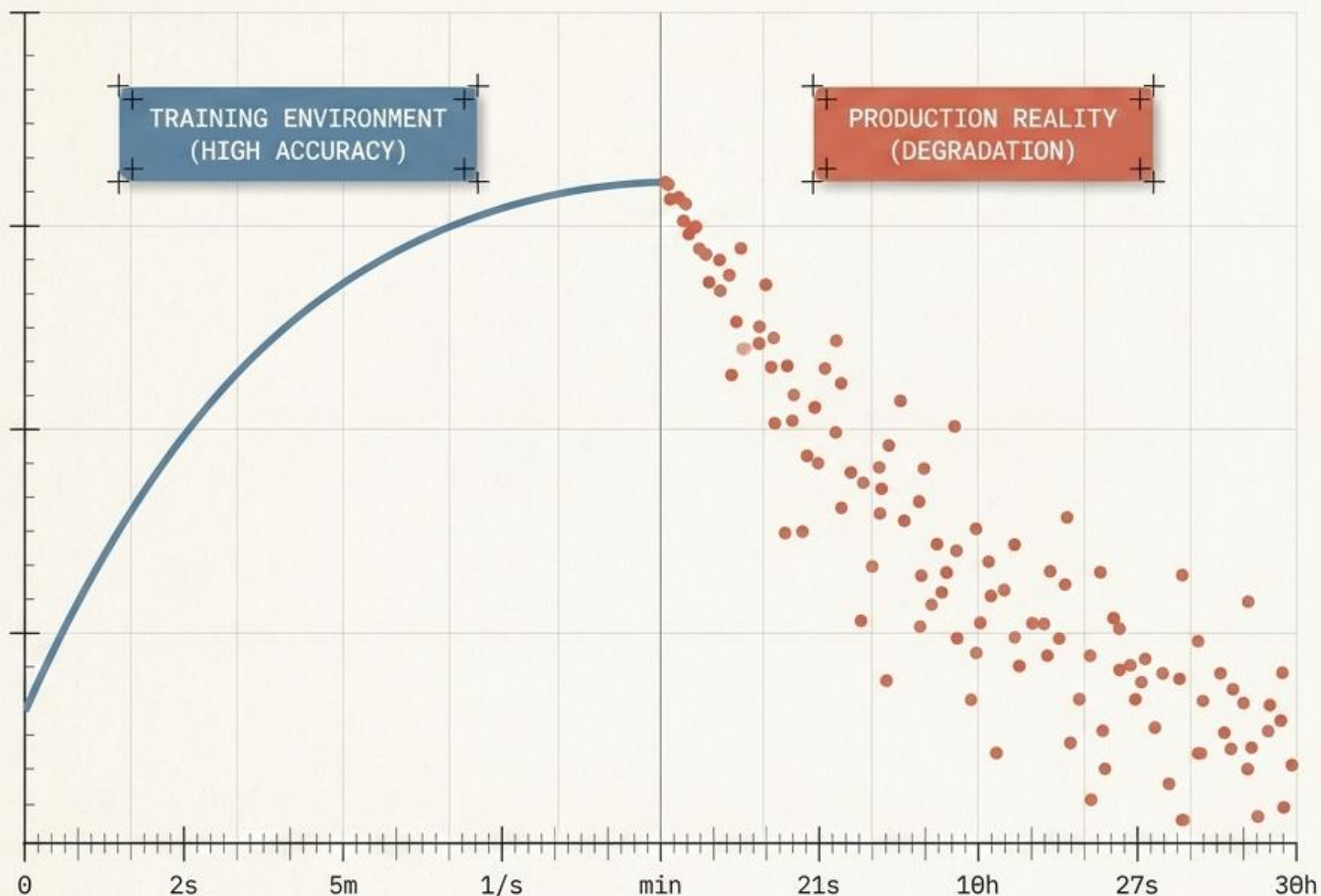
✓	Model Serialization: Exported trained weights using Joblib.
✓	API Wrapper: Configured REST endpoints using FastAPI.
✓	Dependency Locking: Captured exact library versions (requirements.txt).
✓	Containerization: Packaged OS, runtime, and app into a Docker Image.
✓	Cloud Hosting: Deployed to AWS EC2.
✓	Network Access: Configured Security Groups to expose application ports.

Machine Learning Engineering is not just about algorithmic accuracy; it is about designing reproducible, maintainable, and accessible software systems.



Machine learning engineering is not about shipping an artifact.
It is about maintaining a living system in a changing environment.

MODEL DEGRADATION & REALITY GAP



“REAL-WORLD DATA CHANGES OVER TIME. THE MOMENT A MODEL IS DEPLOYED, IT BEGINS TO DEGRADE.”

🔒 THE SILENT FAILURE

- Traditional software bugs are errors in code.
- Machine learning failures are shifts in the environment.
- Monitoring is a necessity, not an afterthought.

What is Data Drift?

 **Data Drift (Covariate Shift):** The statistical distribution of the model's INPUT features ($P(X)$) changes between training time and inference time — but the true relationship $P(Y|X)$ stays the same.

Feature Drift

Individual input columns shift: mean, variance, or shape of distribution changes. E.g., user age distribution skews younger after a new product launch.

Population Drift

The mix of user segments served by the model changes. E.g., model trained on US data now serves global users with very different behavior patterns.

Schema Drift

New features appear, old ones go missing, or data types change. E.g., upstream pipeline adds a new categorical value not seen in training.

What is Concept Drift?

 Concept Drift: The true mapping $P(Y|X)$ changes — the same inputs now produce different correct outputs. The model's learned relationship is no longer valid, even if inputs look similar.



Sudden

Abrupt change overnight.
Example: COVID-19 disrupts demand forecasting models instantly.



Gradual

Slow shift over weeks/months.
Example: Customer tastes evolve, sentiment model loses accuracy.



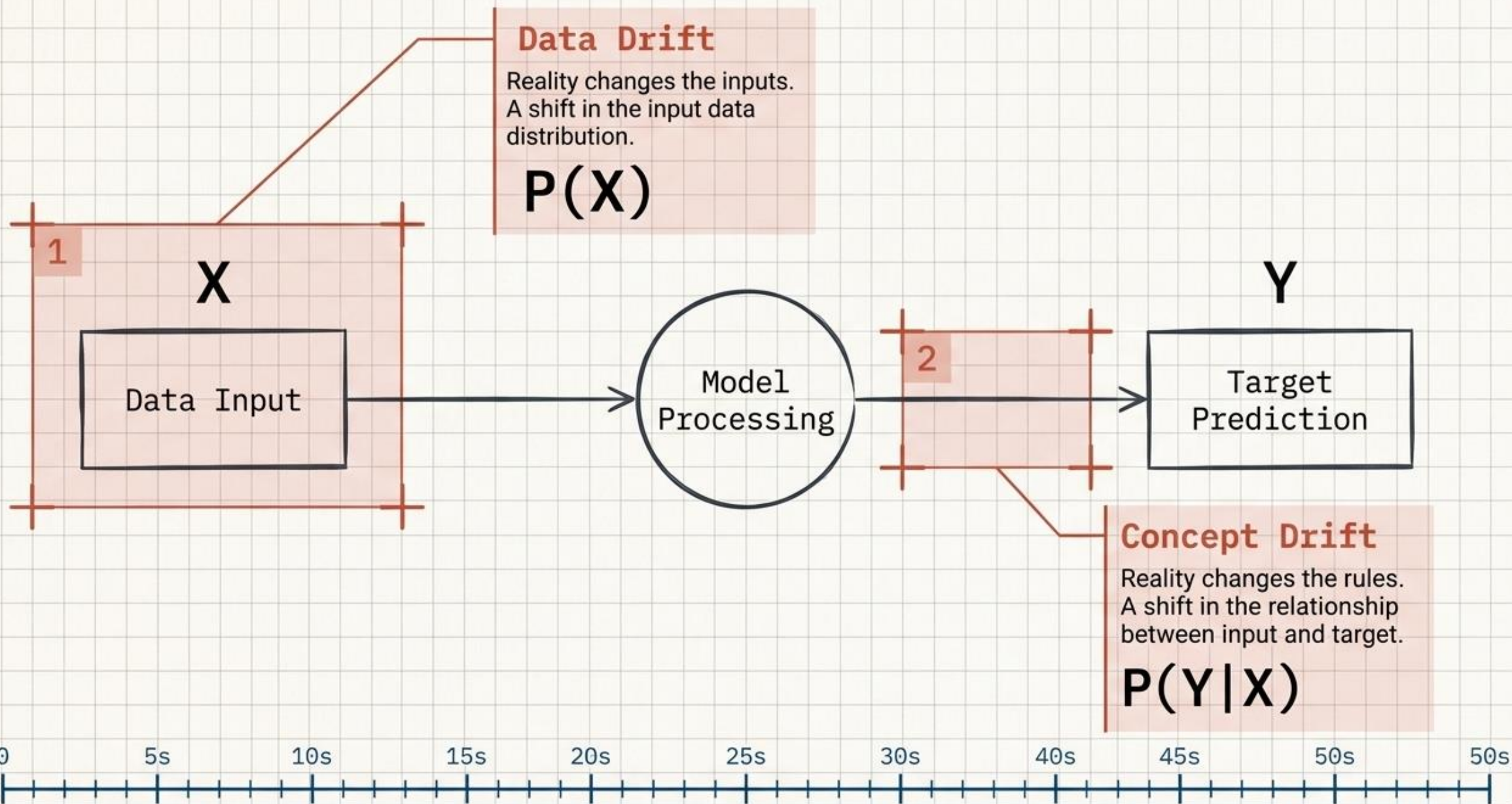
Incremental

Continuous slow drift. Example: Inflation erodes the meaning of price-based fraud thresholds.



Recurring

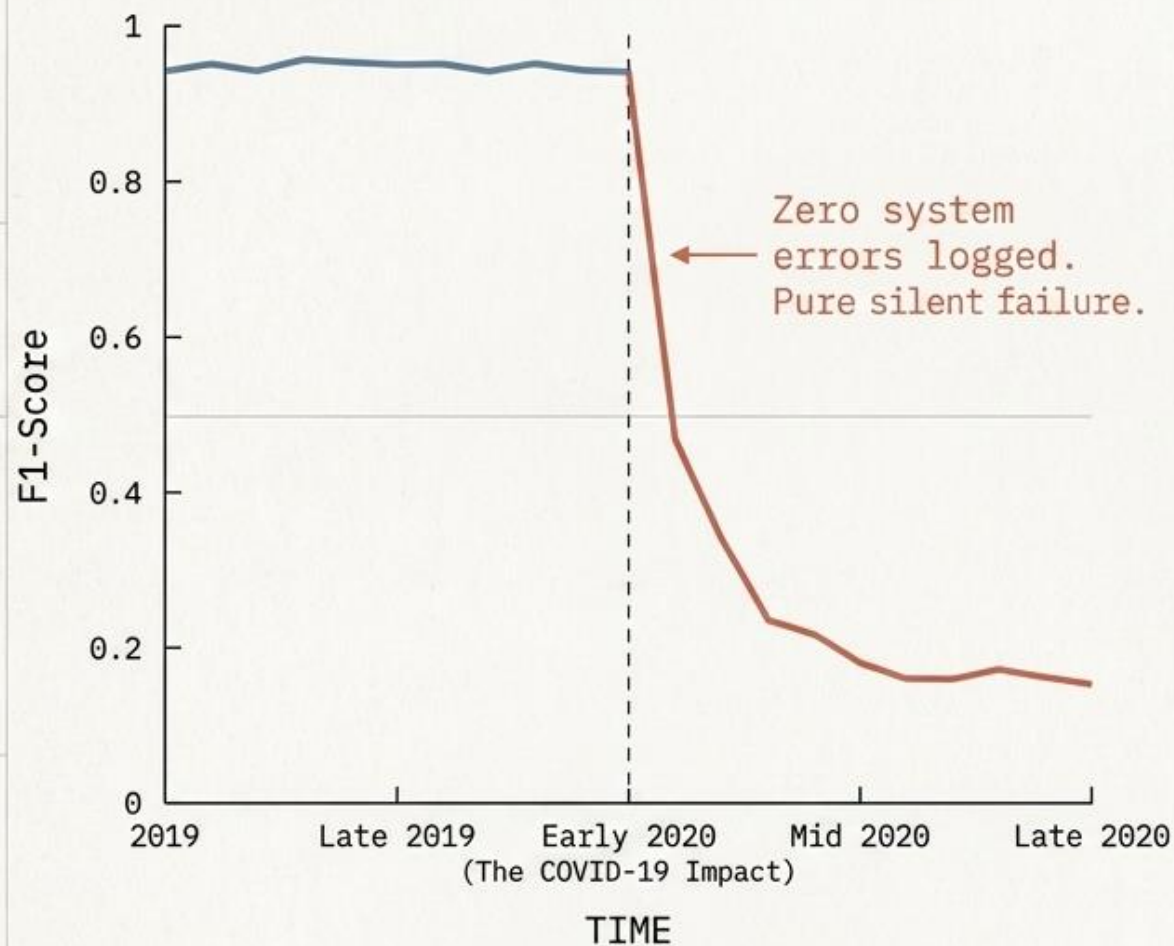
Seasonal or cyclical patterns.
Example: Holiday shopping model needs periodic retraining.



Drift Comparison: Side by Side

Aspect	Data Drift	Concept Drift
What changes?	$P(X)$ — input distribution	$P(Y X)$ — output relationship
Inputs look different?	✅ Yes	❌ Not necessarily
Labels change?	❌ Not directly	✅ Yes — ground truth shifts
Detected by?	Feature statistics, KS-test	Model accuracy metrics, label comparison
Example	New user segments in traffic	"Spam" patterns evolve post-update
Risk level	Medium — model still valid	High — model fundamentally wrong
Action	Monitor & retrain if needed	Immediate retraining or fallback

MODEL F1-SCORE OVER TIME



WHY CONCEPT DRIFT IS HARDER TO CATCH

1. **Requires Ground Truth:** Cannot be detected by looking at inputs alone; requires newly labeled data to confirm.
2. **Silent Failure:** No traditional software bugs or server crashes. Often only discovered via severe downstream performance drops.
3. **Rapid Onset:** Customer behavior can change drastically overnight, breaking models trained on historical assumptions.

MODEL DEGRADATION DETECTED

```
graph TD; A[MODEL DEGRADATION DETECTED] --> B[Is it Data Drift?]; A --> C[Is it Concept Drift?]; B --> D["- Feature Engineering Updates<br/>(Handling new categorical inputs)<br/>- Batch Retraining"]; C --> E["- Continuous Online Learning<br/>(Updating model live)<br/>- Weight Decay (Forcing model<br/>to forget old paradigms)"]
```

Is it Data Drift?

- Feature Engineering Updates
(Handling new categorical inputs)
- Batch Retraining

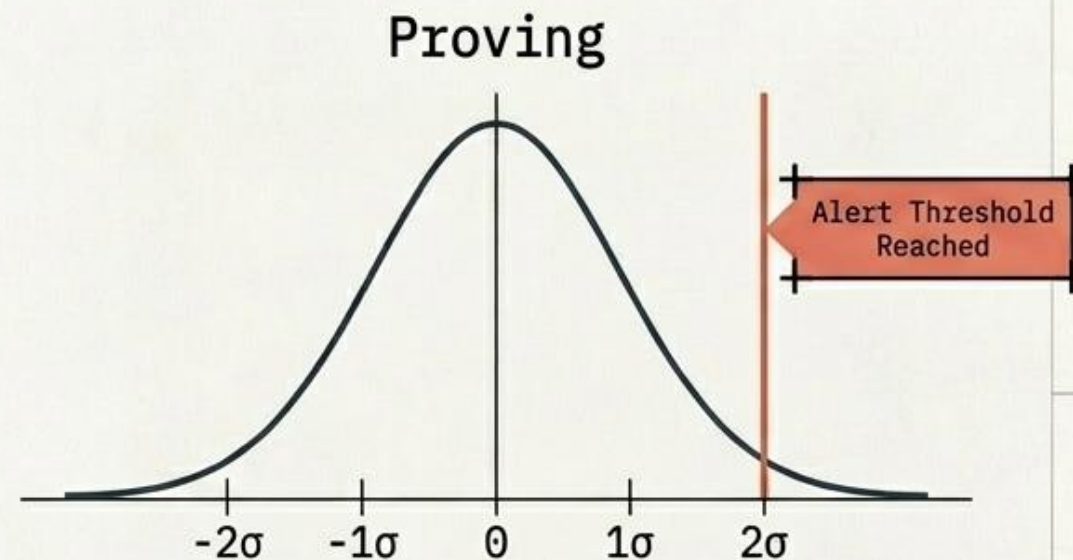
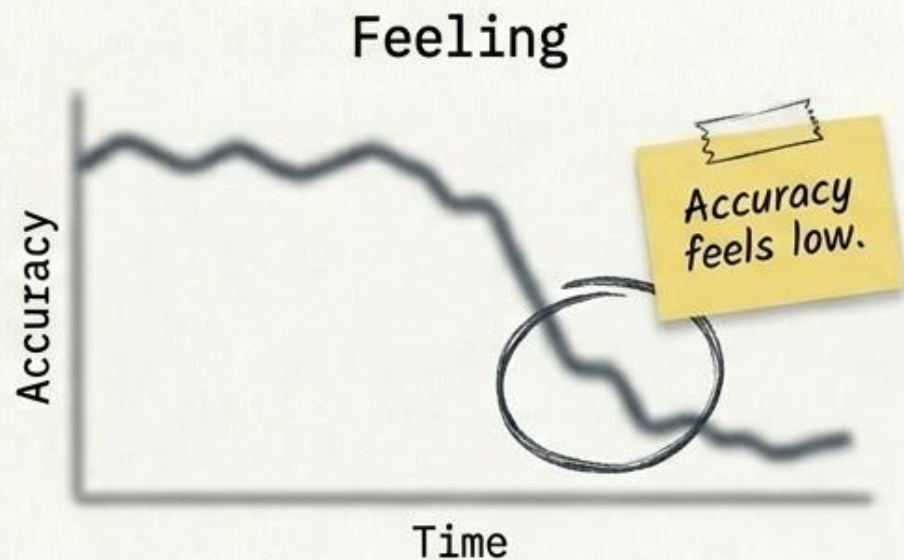
Is it Concept Drift?

- Continuous Online Learning
(Updating model live)
- Weight Decay (Forcing model
to forget old paradigms)

KEY INSIGHT

Not all drift requires throwing away the model. Sometimes, updating a scaling parameter or preprocessing pipeline is enough to restore the $P(X)$ distribution.

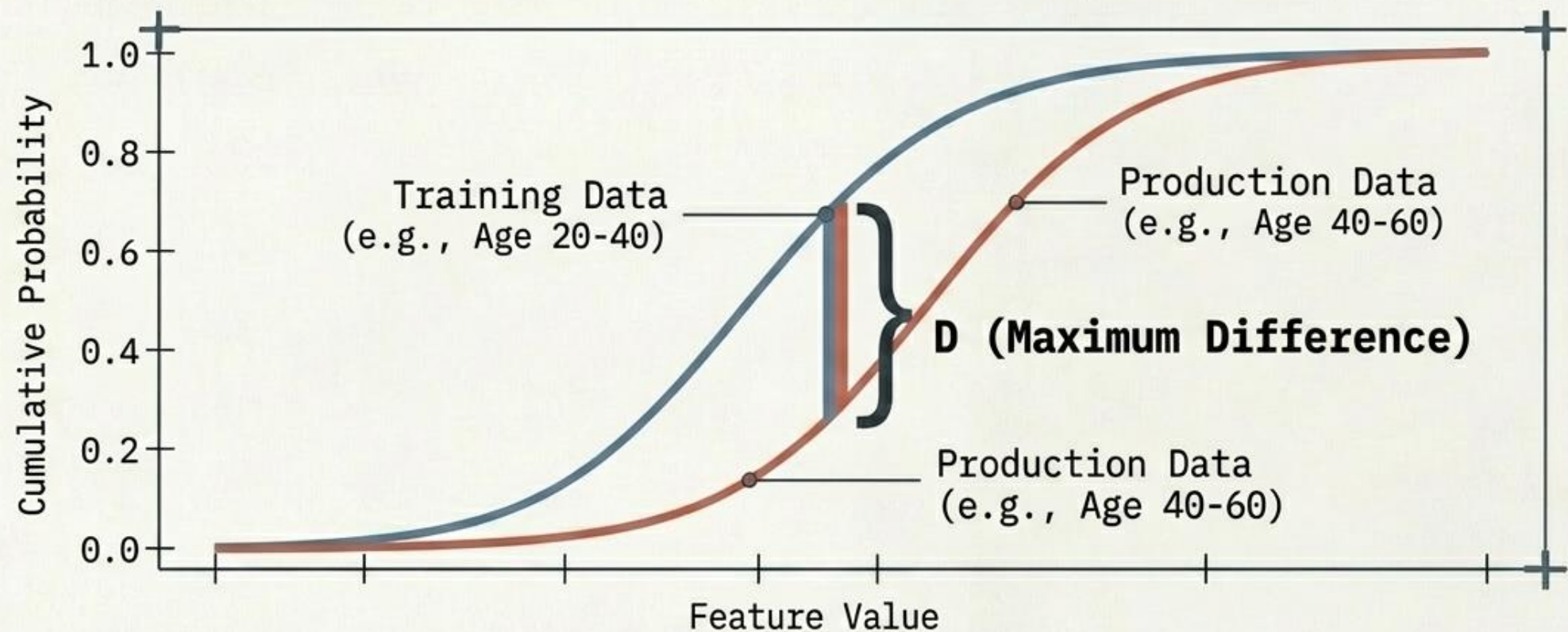
The Need for Mathematics



We cannot rely on 'feeling' that a model is failing, or waiting for business metrics to crash. We need quantitative, statistical evidence of drift before the silent failure impacts the user.

Enter the Statistical Checks ->

The Kolmogorov-Smirnov (KS) Test Explained



The KS-Test compares two continuous distributions by finding the single point where their cumulative probabilities diverge the most.

The Kolmogorov-Smirnov (KS) Statistic Definition

The KS Statistic.
Small D = similar distributions.
Large D = drift detected.

The Cumulative Distribution
Function (CDF) of the
Training Data.

$$D = \sup_x |F_1(x) - F_2(x)|$$

Supremum. The absolute
maximum distance across
all values of x .

The Cumulative Distribution
Function (CDF) of the
Production Data.

Kolmogorov–Smirnov Test: How It Works

KS Statistic

$$D = \sup | F_{\text{train}}(x) - F_{\text{prod}}(x) |$$

Where:

$F_{\text{train}}(x)$ = CDF of training data

$F_{\text{prod}}(x)$ = CDF of production data

$\sup$ = supremum (maximum gap)

$D \in [0, 1]$ = 0 means identical

p-value < 0.05 → distributions significantly different → DRIFT DETECTED

1

Collect Reference

Use a representative sample of training data (or a recent baseline window) as the reference distribution.

2

Compute CDFs

Calculate the empirical CDF of both training and production feature values across the monitoring window.

3

Find Max Gap (D)

Measure the largest vertical distance between the two CDF curves. This is the KS statistic D.

4

Compute p-value

Compare D to KS distribution. Low p-value (<0.05) means the two samples are statistically different.

5

Alert & Act

Trigger alert, log the drifted feature, and schedule retraining or deeper investigation.

KS-Test: Thresholds, Interpretation & Python Code

$D < 0.1$

→ No Drift

Distributions are nearly identical. Model is stable.

$0.1 \leq D < 0.2$

→ Mild Drift

Slight shift. Monitor closely; retraining may be optional.

$D \geq 0.2$

→ Significant Drift

Major shift detected. Investigate root cause, retrain.

```
from scipy import stats
import numpy as np

def ks_drift_check(train_col, prod_col, threshold=0.1):
    stat, p_value = stats.ks_2samp(train_col, prod_col)
    drifted = stat > threshold or p_value < 0.05
    return {"ks_statistic": round(stat, 4), "p_value": round(p_value, 4), "drift": drifted}
```

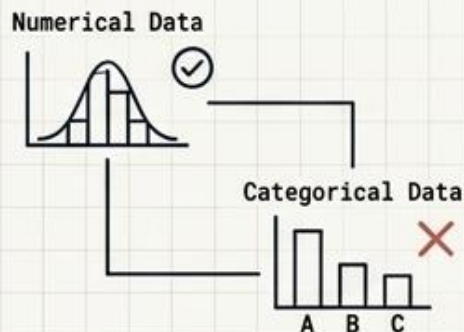
System Limitations & Governance Rules



Feature Constraints

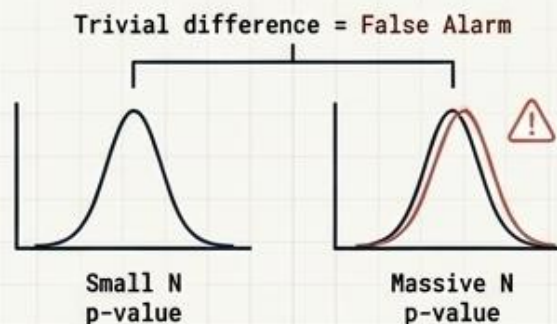
Works strictly for numerical features.

It is fundamentally incompatible with categorical data.



Sample Size Sensitivity

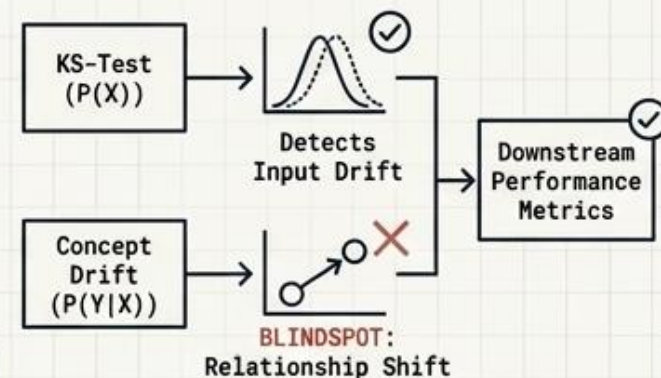
Highly sensitive to sample size. In massive datasets, even trivial, non-impactful differences will trigger a low p-value and a false alarm.



The Concept Drift Blindspot

The KS-Test ONLY checks $P(X)$. It is entirely blind to Concept Drift $P(Y|X)$.

It must be used alongside downstream performance metrics.



Beyond KS-Test: Other Statistical Methods

Chi-Square Test

Use: Categorical features

Tests if observed category frequencies match expected. Use for feature columns with discrete values.

✓ *Best for: Low-cardinality categoricals*

Jensen-Shannon Divergence

Use: Any distribution

Symmetric version of KL-Divergence. Measures similarity between two probability distributions (0=identical, 1=totally different).

✓ *Best for: Robust, bounded metric*

Population Stability Index (PSI)

Use: Credit / banking domain

PSI < 0.1: stable. 0.1–0.25: investigate. > 0.25: major shift. Widely used in risk model monitoring.

✓ *Best for: Scorecard / risk models*

Wasserstein Distance

Use: Continuous, multi-modal

Earth Mover's Distance — how much 'work' to transform one distribution into another. More robust than KS for heavy tails.

✓ *Best for: Complex distributions*

Maximum Mean Discrepancy (MMD)

Use: High-dimensional

Kernel-based test comparing mean embeddings in a feature space. Detects subtle multi-dimensional shift.

✓ *Best for: Image / NLP models*

Anderson-Darling Test

Use: Normality & comparison

More sensitive than KS to differences in tails. Good when outlier behavior matters most.

✓ *Best for: Tail-sensitive use cases*